Full Stack Development with MERN

Project Documentation format

1. Introduction:

- **Project Title:** Smart library system request workflow.
- **Team Members: MUHMED JASSIR .S(TL)**
  KISHOR.L
  NITHISH.M
  MANIKANDAN.S

2. Project Overview:

Purpose: The purpose of the Smart Library System Request Workflow is to automate book requests, approvals, and issue processes. It helps reduce manual work, improve efficiency, and provide quick access to library resources.

Feature: Book Request System Users can request books online.

3. Architecture :

Frontend (React):
React is used to create user interfaces for book requests, status tracking, and admin dashboard.

Backend (Node.js & Express.js):
Node.js and Express.js handle request processing, approvals, and API communication.

Database (MongoDB):
MongoDB stores user details, book information, and request workflow data.

4. Setup Instruction:

Prerequisites:
Node.js, MongoDB, Git, and Code Editor must be installed.

Installation:
Clone the repository, install dependencies using `npm install`, configure environment variables, and run the project using `npm start`.

5.Folder Structure:

Client (React):
Contains components, pages, services, and assets folders for managing UI and user interactions.

Server (Node.js):
Includes routes, controllers, models, and configuration files to handle APIs and database operations.

6. Running the Application :

Frontend:
Run `npm start` in the client directory to start the React frontend.

Backend:
Run `npm start` in the server directory to start the Node.js backend server.

7. API Documentation:

Endpoints Documentation:
All backend endpoints such as book request, approval, and user management are documented.

Request & Response Details:
Each API includes request method, parameters, and example responses for proper usage.

8. Authentication :
User authentication is handled using login credentials, and authorization controls access based on user roles such as admin or user.

Token Management:
JWT tokens are used to maintain user sessions and secure API access.

9. User Interface: Frontend Design
The user interface is developed using React to provide a responsive and interactive experience.

User Interaction:
Users can request books, view request status, and manage approvals through simple and user-friendly dashboard screens.

10. Testing :

Unit Testing:
Tests individual components and functions of the system.

Integration Testing:
Tests interaction between frontend, backend, and database.

11. Screenshots or Demo: **Screenshots:**
Displays images of key features such as book request, approval, and dashboard screens.

Demo:
Provides a demonstration of system functionality and workflow of the smart library system.

12. Known Issues:

performance Limitations:

System may slow down when handling large amounts of data.

Network Dependency:
Application requires internet connection for proper functionality.

Feature Limitations:
Some advanced features may be under development or not fully implemented.

13. Future Enhancements:

Mobile App: Easy mobile access